Docker & CI/CD Research & Development Document

Project: IT Resource & Operations Platform (IROP) **Version:** 1.0 **Date:** November 2026 **Author:** IROP Development Team **Document Type:** R&D / Technical Decision Record

Table of Contents

1. Why Docker?
2. Container Architecture
3. Dockerfile Strategy
4. Docker Compose
5. CI Pipeline ΓÇö GitHub Actions
6. Docker Publish Pipeline
7. Registry Choice
8. Deployment Options
9. Security Considerations
10. Risks & Mitigations
11. Final Decisions Summary

1. Why Docker?

1.1 Problem Statement

Without containerisation, IROP faces these development and deployment challenges:

Challenge	Impact
"Works on my machine" syndrome	Differences in Node.js versions, OS-level dependencies, and MongoDB versions cause inconsistent behaviour
Complex local setup	Reviewers must install Node.js, MongoDB, and configure multiple `.env` files manually
Environment drift	Development, testing, and production environments diverge over time
Deployment friction	Manual deployment processes are error-prone and hard to reproduce

1.2 How Docker Solves These

Docker Benefit	Application to IROP
Consistency	Every developer, reviewer, and CI runner uses the exact same Node.js 20, nginx, and MongoDB versions packaged in the images
Isolation	Each service (client, server, database) runs in its own container with its own filesystem, process space, and network interface
Reproducibility	`docker compose up --build` produces an identical environment on any machine with Docker installed ΓÇö no other prerequisites
Single-command startup	Reviewers can launch the entire stack (client + server + MongoDB) with one command and begin evaluating immediately
CI/CD integration	Docker images can be built and tested in CI, then published to a registry for deployment without rebuilding

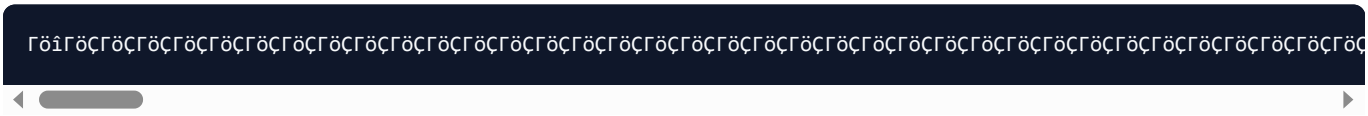
1.3 Alternatives Considered

Alternative	Verdict	Reason
Bare metal / manual setup	Γ¥î Rejected	Requires Node.js + MongoDB installation; version mismatches are inevitable
Vagrant	Γ¥î Rejected	Heavier than Docker (full VMs); slower startup; overkill for a 3-service app
Podman	ΓÜá┐ Å Viable but deferred	Compatible with Docker CLI; less ecosystem tooling; Docker is the industry standard
Docker	Γ£à Selected	Ubiquitous, lightweight, excellent tooling, native GitHub Actions support

---

2. Container Architecture

IROP runs as three Docker containers orchestrated by Docker Compose:



Service Responsibilities

Service	Base Image	Role	Exposed Port
`client`	`nginxinc/nginx-unprivileged`	Serves the Vite-built React SPA; nginx handles SPA fallback routing to `index.html`	8080
`server`	`node:20-alpine`	Runs the Express API; connects to MongoDB; serves all `/api/*` and `/health` endpoints	5000
`mongo`	`mongo:7` (official)	Provides the MongoDB database; data persisted via a named Docker volume	27017 (internal)

Communication Flow

1. Browser ΓåÆ `client:8080` ΓÇö serves static HTML/CSS/JS
2. Client JS ΓåÆ `server:5000` ΓÇö API requests via Axios (base URL configured at build time)
3. Server ΓåÆ `mongo:27017` ΓÇö Mongoose connection using Docker's internal DNS (`mongodb://mongo:27017/irop`)

---

3. Dockerfile Strategy

3.1 Multi-Stage Builds

Both the client and server Dockerfiles use **multi-stage builds** to minimise final image size and exclude build-time dependencies (TypeScript compiler, dev dependencies, source `.ts` files).

3.2 Server Dockerfile (`server/Dockerfile`)

```
# Stage 1: Install dependencies FROM node:20-alpine AS deps WORKDIR /app COPY package*.json ./ RUN npm ci  # St
```

Key decisions:

Decision	Rationale
`node:20-alpine`	Minimal image (~50 MB vs ~350 MB for `node:20`); reduces attack surface
Multi-stage	Build artifacts only; no TypeScript compiler or `devDependencies` in runtime
`npm prune --production`	Removes dev dependencies from the runtime image
Non-root user (`appuser`)	Security best practice; limits damage if the container is compromised
`HEALTHCHECK`	Docker and Compose can monitor container health; auto-restart on failure

3.3 Client Dockerfile (`client/Dockerfile`)

```
# Stage 1: Build the SPA FROM node:20-alpine AS build WORKDIR /app COPY package*.json ./ RUN npm ci COPY . . AR
```

Key decisions:

Decision	Rationale
`nginxinc/nginx-unprivileged`	Runs as non-root by default (UID 101); no need for `USER` directive
Build-time `ARG VITE_API_URL`	Vite inlines env vars at build time; the API URL is baked into the JS bundle
SPA fallback in `nginx.conf`	`try_files $uri $uri/ /index.html` ensures client-side routing works
No Node.js in runtime	The final image contains only static files and nginx (~25 MB)

3.4 `.dockerignore`

Both services include a `.dockerignore` to keep the build context small and avoid leaking secrets:

```
node_modules dist .env .env.* .git coverage tests *.log
```

4. Docker Compose

4.1 Service Definition (`docker-compose.yml`)

```
version: "3.9"  services:   mongo:     image: mongo:7     volumes:       - mongo_data:/data/db     healthcheck:
```

4.2 Key Design Choices

Aspect	Choice	Rationale
Named volume (`mongo_data`)	Persistent storage for MongoDB	Data survives `docker compose down` ; only `docker compose down -v` removes it
Health checks on `mongo`	`mongosh --eval "db.adminCommand('ping')"`	Server waits for a healthy database before starting; prevents connection errors on cold start
`depends_on` with condition	`service_healthy` for mongo	Ensures startup ordering; server won't start until MongoDB is responsive
Network (`irop` bridge)	Dedicated bridge network	Containers communicate by service name (e.g., `mongodb://mongo:27017`); isolated from host network
No secrets in Compose file	`JWT_SECRET` sourced from env or `.env` file	The compose file can be committed; actual secrets stay in git-ignored `.env` files
Pointing to Atlas	Override `MONGODB_URI` in `.env`	For cloud deployment, simply change the URI; the mongo service can be removed or ignored

4.3 Usage

```
# Build and start all services docker compose up --build  # Stop services (data preserved) docker compose down
```

5. CI Pipeline ΓÇö GitHub Actions

5.1 Pipeline Design (`.github/workflows/ci.yml`)

The CI pipeline runs on every push and pull request to ensure code quality before merge.

```
name: CI  on:   push:     branches: [main, develop]   pull_request:     branches: [main]  concurrency:   group:
```

5.2 Pipeline Design Decisions

Decision	Rationale
Matrix strategy (`server` , `client`)	Both projects run in parallel, cutting CI time roughly in half
Service container for MongoDB	Server tests need a real MongoDB instance; the GitHub Actions service container provides one without external dependencies
`irop_test` **database**	Tests use a separate database name to avoid conflicts
No Atlas in CI	CI uses a local service container; no cloud credentials needed; safe for forks and public PRs
`concurrency` **with cancel-in-progress**	Stale pipeline runs are automatically cancelled when new commits are pushed, saving runner minutes
Pinned action versions (`@v4`)	Pinned to major versions for stability; minor updates are automatic
npm cache	`actions/setup-node` caches `node_modules` based on `package-lock.json` ; speeds up subsequent runs by ~30 seconds

5.3 CI Steps Breakdown

```
ГöîГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÉ  ГöîГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÉ  ГöîГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÇГöÉ  ГöîГöÇ
```

Each step is a gate: if lint fails, subsequent steps don't run, providing fast feedback.

---

6. Docker Publish Pipeline

6.1 Pipeline Design (`.github/workflows/docker-publish.yml`)

```
name: Docker Publish  on:   push:     branches: [main]     tags: ["v*.*.*"]  concurrency:   group: docker-${{ g
```

6.2 Publish Design Decisions

Decision	Rationale
Triggers: `main` branch + `v*.*.*` tags	Images are published on every merge to main (for latest builds) and on tagged releases (for versioned deployments)
`docker/metadata-action`	Automatically generates semantic tags (`v1.2.3` , `v1.2` , `sha-abc1234` , `main`); follows OCI best practices
`docker/build-push-action`	Industry-standard action; supports multi-platform builds, caching, and push-to-registry in a single step
`GITHUB_TOKEN` **authentication**	No additional secrets needed; the built-in token has `packages:write` permission when declared
Matrix for server + client	Both images build and push in parallel; each gets its own GHCR package
Runs after CI	The Docker publish workflow runs independently but only on main/tags (CI must pass before merge)

6.3 Tag Strategy

Event	Generated Tags
Push to `main`	`main` , `sha-<commit>`
Tag `v1.2.3`	`1.2.3` , `1.2` , `sha-<commit>`

---

7. Registry Choice

7.1 Options Evaluated

Registry	Auth	Cost	Integration	Verdict
Docker Hub	Separate account + access token	Free tier: 1 private repo	Requires additional secrets setup	Γ¥î
GitHub Container Registry (GHCR)	Built-in `GITHUB_TOKEN`	Free for public repos; generous free tier for private	Native integration with GitHub Actions, permissions, and packages	Γ£à Selected
AWS ECR	AWS IAM credentials	Free tier: 500 MB/month	Requires AWS account setup	Γ¥î (overkill)
Self-hosted	Custom	Free	Full control but maintenance overhead	Γ¥î (academic scope)

7.2 Why GHCR

1. **Zero additional secrets:** `GITHUB_TOKEN` is automatically available in every Actions workflow ΓÇö no manual credential management.
2. **Native GitHub integration:** Published packages appear in the repository's Packages tab; visibility follows repo visibility.
3. **OCI-compliant:** Supports standard Docker image manifests and tags.

4. **Free for public repositories:** No storage or bandwidth costs for open-source projects.
5. **Permissions model:** `packages:write` permission is declarative in the workflow YAML; no external IAM configuration.

---

8. Deployment Options

8.1 Primary: Local Docker Compose (Recommended for Review)

```
git clone <repo-url> cd irop docker compose up --build # Visit http://localhost:8080
```

Pros	Cons
Single command	Requires Docker installed
Fully self-contained	Not publicly accessible
No cloud accounts needed	Reviewer must have hardware resources
Matches production topology	ΓÇö

8.2 Optional: Cloud Deployment (Live Demo)

For a publicly accessible demo, the following free-tier stack can be used:

Component	Service	Tier	Notes
Server	Render (Web Service)	Free	Sleeps after 15 min inactivity; ~30s cold start
Client	Vercel or Netlify	Free	Global CDN; instant deploys from GitHub
Database	MongoDB Atlas	M0 Free	512 MB storage; shared cluster; IP allowlist required

Cloud deployment considerations:

Consideration	Mitigation
Render free tier sleeps	Add a "LoadingΓÇª" indicator on first request; document the cold start in README
Atlas requires IP allowlist	Use `0.0.0.0/0` for demo (with a dedicated demo DB user with minimal privileges); document the trade-off
CORS configuration	Set `CLIENT_ORIGIN` to the deployed client URL (e.g., `https://irop.vercel.app`)
Environment variables	Configure via each platform's dashboard; never commit to repo
Separate from local DB	Use a dedicated `irop-demo` database; seed data separately

8.3 Deployment Decision

Primary deployment is local Docker Compose. Cloud deployment is documented as optional and may be configured for live demonstrations. The Docker Compose setup is the authoritative containerized local production

replica.

---

9. Security Considerations

9.1 Container Security

Measure	Implementation
Non-root containers	Server runs as `appuser` (UID 1001); client uses `nginx-unprivileged` (UID 101). No container runs as root.
Minimal base images	`node:20-alpine` (~50 MB) and `nginx-unprivileged:alpine` (~25 MB) minimise attack surface and known CVE count.
No secrets in layers	Environment variables are injected at runtime via Docker Compose `environment` or `.env` files. No `ENV JWT_SECRET=...` in Dockerfiles.
`.dockerignore`	Excludes `.env`, `.git`, `node_modules`, `coverage`, and test files from the build context.
Dependency audit	`npm audit --omit=dev` runs as a non-blocking CI step; critical vulnerabilities are reviewed before release.

9.2 Application Security

Measure	Implementation
Helmet.js	Sets security headers: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Strict-Transport-Security`, `X-XSS-Protection`.
Rate limiting	`express-rate-limit` on the login endpoint to mitigate brute-force attacks (e.g., 10 attempts per 15 minutes per IP).
JWT security	Tokens signed with a strong secret from env vars; short expiry (e.g., 24h); payload contains only `id` and `role`.
bcrypt	Passwords hashed with cost factor ГёЙ 10; never stored or transmitted in plain text.
Input validation	Zod schemas validate all request bodies before processing; rejects malformed input at the boundary.
CORS	Configured with explicit `CLIENT_ORIGIN`; no wildcard `*` in production.
Error handling	Generic error messages for auth failures; no stack traces, internal paths, or hashes in responses.
Mongoose	Parameterised queries prevent NoSQL injection; `select('-passwordHash')` excludes sensitive fields.

9.3 JWT Storage Trade-Off

> **Decision:** Store the JWT in `localStorage` + in-memory state.

Approach	Pros	Cons
`localStorage` (selected)	Simple implementation; persists across page refreshes; works with any API	Vulnerable to XSS (malicious script can read the token)
`httpOnly` **cookie** (production best practice)	Immune to XSS (JavaScript cannot access the cookie)	Requires server-side cookie management, CSRF protection, and `SameSite` configuration

Rationale: In this release, `localStorage` enables portable REST client testing and simplifies client-side state hydration while demonstrating full JWT validation. In enterprise production deployments, `httpOnly` cookies with `SameSite=Strict` can be adopted.

9.4 MongoDB Security

Measure	Implementation
Local dev	No authentication on the local Docker MongoDB (acceptable for development; container is not exposed to the internet)
Atlas (if used)	Dedicated demo user with `readWrite` on the `irop` database only; IP allowlist; TLS enforced
No admin credentials in code	Atlas connection strings with credentials are stored in environment variables, never committed

---

10. Risks & Mitigations

#	Risk	Likelihood	Impact	Mitigation
R-1	Docker not installed on reviewer's machine	Medium	High (cannot run the app)	Provide clear installation link in README; document fallback: run `npm run dev` in server/ and client/ with a local MongoDB
R-2	MongoDB container fails to start (port conflict)	Low	Medium	Health check + `depends_on: condition: service_healthy`; document port change instructions
R-3	Render free tier cold start delays	High	Low (demo only)	Document the expected 30s delay in README; add a loading indicator on the client
R-4	GitHub Actions runner minutes exhausted	Low	Medium	`concurrency` cancels stale runs; matrix strategy keeps total time low; free tier provides 2,000 min/month
R-5	Dependency vulnerabilities (npm audit)	Medium	Medium	`npm audit` in CI (non-blocking); review critical CVEs; use `--omit=dev` in production images
R-6	JWT secret leaked in logs or error messages	Low	Critical	Secret sourced from env var; error handler strips sensitive data; no `console.log` of env vars
R-7	XSS stealing JWT from localStorage	Low	High	Documented trade-off; CSP headers via Helmet; in production, migrate to httpOnly cookies
R-8	Docker image size bloat	Low	Low	Multi-stage builds; Alpine images; `.dockerignore`; `npm prune --production`
R-9	CI tests flaky due to MongoDB timing	Low	Medium	Health check on service container; retry logic in test setup; `start_period` on health checks
R-10	Atlas free tier IP restrictions block CI	Medium	Medium	CI uses service container MongoDB, not Atlas; Atlas is only for optional live demo

11. Final Decisions Summary

Area	Decision
Containerisation	Docker with multi-stage builds for both client and server
Base images	`node:20-alpine` (server), `nginxinc/nginx-unprivileged:alpine` (client), `mongo:7` (database)
Orchestration	Docker Compose v3.9 with health checks, named volumes, and a dedicated bridge network
CI pipeline	GitHub Actions on push/PR: lint Γåá typecheck Γåá build Γåá test (matrix: server + client)
Docker publish	GitHub Actions on main/tags: build and push to GHCR using `metadata-action` + `build-push-action`
Container registry	GitHub Container Registry (GHCR) with `GITHUB_TOKEN` authentication
Primary deployment	Local `docker compose up --build` for standardized local evaluation
Optional deployment	Render (server) + Vercel/Netlify (client) + MongoDB Atlas M0 (database)
Security posture	Non-root containers, Helmet, rate limiting, bcrypt, Zod validation, no secrets in layers
JWT storage	`localStorage` (portable client trade-off; documented; production can use httpOnly cookies)
MongoDB in CI	GitHub Actions service container (no Atlas dependency; safe for forks)
Concurrency	Cancel-in-progress on both CI and Docker publish workflows
Dockerfiles committed	Yes. Built images are never committed to the repository.

End of Docker & CI/CD Research & Development Document